

**UNIVERSIDAD ESTATAL PENÍNSULA DE
SANTA ELENA**

**FACULTAD DE SISTEMAS Y TELECOMUNICACIONES
CARRERA DE INGENIERIA EN SOFTWARE**



ASIGNATURA:
CALIDAD DE SOFTWARE

TEMA:
PLAN DE GESTIÓN DE CALIDAD DEL PROYECTO

ELABORADO POR:

- ANDY BRYAN ALEJANDRO VERA
- ALISSON YAMEL REYES RICARDO
- YANDRIS MIGUEL RIVERA TORRES

CURSO Y PARALELO:
SOFTWARE 6/1

DOCENTE:
ING. ANTHONY ABRAHAN PACHAY ESPINOZA

LA LIBERTAD – ECUADOR

Plan de Gestión de Calidad de Software

Sistema Web de Gestión de incidencias Georreferenciadas

1.	Propósito y Alcance.....	2
a.	Propósito	2
b.	Objetivo General del sistema	2
c.	Alcance del Plan	3
d.	Alcance del sistema a validas	3
e.	Usuarios involucrados.....	3
2.	Referencias Normativa	3
a.	Estándares	4
b.	OWASP	4
3.	Roles y Responsabilidades	6
a.	Roles de calidad asignados	6
b.	Distribución de Responsabilidades de Calidad	6
c.	Responsabilidades Compartidas	7
4.	Objetivo.....	7
a.	Objetivos estratégicos de calidad:	7
b.	Objetivos Operativos.....	7
5.	Atributos de calidad	8
6.	Plan de Testing Funcional	9
a.	Formato Estructurado	9
b.	Ejemplo de nomenclatura	9
c.	Principio de validación Dual.....	10
7.	Matriz de caso de prueba	10
a.	Modulo 01: Gestión de incidencias.....	11
b.	Modulo 02: Estados e historial	13
c.	Módulo 03: Asignación responsable	15
d.	Modulo 04: Sistemas de comentarios.....	17
e.	Modulo 05: Ubicación Georreferenciada.....	18
f.	Modulo 06: Clasificación Jerarquica.	20
g.	Modulo 07: Sistemas de notificaciones	21

h.	Módulo 08: Dashboard y Métricas	23
i.	Módulo 09: Autenticación y Control de acceso	25
j.	Modulo 10: Validaciones de formato y tipo de datos	27
8.	Resumen de distribución de pruebas	29
a.	Conteo por integrante y capa	29
b.	CP ´s por módulo	29
9.	Plan de ejecución Pruebas	30
a.	Cronograma Semanal	30
b.	Criterios de progreso.....	31
10.	Plantilla de Registro De ejecución de casos de prueba	31
a.	Formato de registro.....	31
b.	Leyenda de severidad.....	31
11.	Criterios De Cierre de plan de calidad	32
12.	Checklist Pre-Sustentación.....	32
13.	Metricas e indicadores del sistema.....	33
14.	Riesgos de calidad y acciones mitigadoras.....	34
15.	Herramientas.....	35

1. Propósito y Alcance

a. Propósito

Este Plan de Calidad establece el marco de trabajo para garantizar que el sistema cumpla con todos los requisitos funcionales, de rendimiento y de calidad. El objetivo principal es prevenir defectos, validar funcionalidades y documentar evidencias de manera sistemática a lo largo de todo el desarrollo.

b. Objetivo General del sistema

Desarrollar una plataforma web que permita a ciudadanos y operadores institucionales registrar, clasificar, georreferenciar, asignar, dar seguimiento y reportar incidencias urbanas o institucionales sobre un mapa interactivo, garantizando trazabilidad del ciclo de vida (Pendiente, En Proceso, Resuelto, Cerrado) y produciendo métricas operativas para la toma de decisiones.

c. Alcance del Plan

Componente	Tecnología	Rol Responsable
Frontend (UI/UX)	HTML, CSS, Bootstrap	Andy Alejandro
Backend (API REST)	Laravel	Allison Reyes
Base de Datos	MySQL/PostgreSQL	Yandris Rivera
Deploy	Docker	Yandris Rivera

d. Alcance del sistema a validas

El plan cubre la validación de las siguientes funcionalidades:

- Gestión de incidencias (CRUD completo).
- Estados e Historial de cambios.
- Asignación de responsables.
- Sistemas de comentarios.
- Ubicaciones georreferenciadas.
- Clasificación jerárquica (Tipo/subtipos).
- Notificaciones.
- Dashboards y métricas.
- Consultas y filtros.

e. Usuarios involucrados

- **Ciudadano/Reportante:** registra incidencias desde el frontend público, sin permisos administrativos.
- **Operador/técnico:** gestiona incidencias asignadas, cambia estados, agrega comentarios y evidencias.
- **Supervisor/Administrador:** gestionar usuarios, roles, catálogos (tipos, subtipos, ubicaciones) y consulta dashboards.
- **Auditor/Visulizador:** acceso de solo lectura a dashboards, métricas e historial.

2. Referencias Normativa

a. Estándares

Estándar	Descripción
ISO/IEC 25000 (SQuaRE)	Marco de referencia para la evaluación de características de calidad de producto de software
ISO/IEC 25010	Define el modelo de calidad: características y sub-características evaluables (Adecuación Funcional, Fiabilidad, Eficiencia de Rendimiento, Portabilidad)
ISO/IEC 29119	Estándar internacional para pruebas de software, incluyendo diseño de casos de prueba
IEEE 730	Lineamientos para la elaboración de Planes de Aseguramiento de Calidad del Software (SQAP)
IEEE 829	Estándar para documentación de pruebas de software
PSR-12	Guía de estilos de codificación para PHP adoptada por la comunidad Laravel

b. OWASP

OWASP Top 10:2025 (publicado en Global AppSec DC, noviembre 2025; vigente desde enero 2026) es el marco de referencia obligatorio para identificar y mitigar los

riesgos críticos de seguridad en la aplicación web. Se evaluarán formalmente los siguientes vectores:

Vector	Nombre	Aplicación en el proyecto
A01:2025	Broken Access Control	Middleware de roles en Laravel (Policies & Gates); pruebas CP-09-*
A02:2025	Security Misconfiguration	Revisión de .env, permisos de storage/, cabeceras HTTP, configuración CORS
A03:2025	Software Supply Chain Failures	composer audit, npm audit, verificación de dependencias Laravel
A05:2025	Injection	Form Requests, Eloquent (prepared statements), sanitización XSS
A07:2025	Authentication Failures	Hash de contraseñas (bcrypt), expiración de tokens, protección de credenciales
A10:2025	Exceptional Condition Handling	Manejo de errores 4xx/5xx, timeouts del mapa, fallback de geocoding

3. Roles y Responsabilidades

a. Roles de calidad asignados

Rol	Área de Responsabilidad	Validaciones
Revisión técnica	Inspeccionar código y validar coherencia arquitectónica	Walkthroughs de arquitectura, code review, validación de estándares.
Diseño de Pruebas	Crear escenarios y especificar casos de prueba bajo IEEE 29119	Análisis de historias de usuario, especificación CP-XX-YY-Z, matrices por módulo
Ejecución de Pruebas	Operar scripts de pruebas funcionales y de rendimiento	Pruebas funcionales manuales, ejecución en Postman, JMeter/k6 para carga
Gestión de Defectos	Administrar el ciclo de vida del defecto	Registro, clasificación por severidad, asignación, re-test y cierre
Elaboración de Métricas	Calcular, monitorear y graficar indicadores de calidad	Cálculo de fórmulas de Sección 8, dashboards y tabla de métricas finales

b. Distribución de Responsabilidades de Calidad

Para un equipo de 3 personas, definimos la siguiente distribución clara de responsabilidades:

Rol	Área de Responsabilidad	Validaciones
Frontend	Validaciones en formularios (HTML5 + JS), verificación de responsividad, pruebas de integración visual, capturas del sistema, dashboard	CP-XX-F (Frontend)
Backend	Validaciones de API con Postman, manejo de errores HTTP, lógica de	CP-XX-B (Backend)

	estados, endpoints REST, métricas de rendimiento	
Base de Datos	Integridad referencial, normalización 3FN, pruebas de consultas SQL, backup y persistencia en Docker, triggers	CP-XX-BD (Base de Datos)

c. Responsabilidades Compartidas

Todas las actividades de testing funcional, evidencias y documentación deben ser realizadas en conjunto por los 3 integrantes.

4. Objetivo

a. Objetivos estratégicos de calidad:

- **Integridad de la información:** Garantizar que los datos registrados y las coordenadas geográficas no sufran pérdidas ni corrupción bajo concurrencia.
- **Minimización de defectos funcionales:** Minimizar la presencia de defectos funcionales en los módulos críticos del sistema.
- **Tiempo de respuesta adecuados:** Hay que asegurar que el sistema responda de forma fluida bajo condiciones operativas normales.
- **Mitigación de riesgos de seguridad:** Aplicar controles básicos de seguridad web sobre accesos, autenticación, configuración, dependencias e inyección.
- **Mantenibilidad del código:** Mantener una arquitectura y estructura de código fuente altamente mantenible que permita la evolución iterativa.

b. Objetivos Operativos

Cada objetivo se mapea directamente con los criterios de evaluación:

#	Objetivo	Criterio Rúbrica	Característica ISO 25010
O1	Validaciones del sistema	Implementación de validaciones y manejo de errores en frontend y backend	Adecuación Funcional — Corrección
O2	Casos de prueba funcionales	Diseño y ejecución de pruebas sobre funcionalidades principales	Adecuación Funcional — Completitud

O3	Evidencias de testing	Capturas, resultados o registros de pruebas realizadas	Fiabilidad — Madurez
O4	Pruebas de carga o estrés	Ejecución básica de pruebas sobre funcionalidades relevantes	Eficiencia de Rendimiento
O5	Uso de herramientas de calidad	Postman, JMeter, DevTools, Laravel Pint, Docker	Mantenibilidad — Analizabilidad
O6	Métricas e indicadores	Datos cuantificables del funcionamiento y calidad del sistema	Adecuación Funcional — Compleitud

5. Atributos de calidad

Atributo de calidad	Justificación Técnica del proyecto
Usabilidad	El sistema posee un enfoque ciudadano y operativo.
Seguridad	Se gestionan accesos restringidos, roles específicos y datos operativos críticos de incidencias urbanas o institucionales.
Fiabilidad	La información capturada y las coordenadas geográficas no deben sufrir pérdidas ni corrupción durante los procesos de concurrencia.
Mantenibilidad	Al ser un proyecto iterativo, la estructura de código debe permitir la evolución continua y la integración de nuevos módulos sin degradar el sistema.
Eficiencia de desempeño	El sistema debe responder de manera fluida y adecuada ante múltiples

	consultas simultáneas sobre la cartografía o mapas.
--	---

6. Plan de Testing Funcional

a. Formato Estructurado

Cada caso de prueba sigue el formato: CP-XX-YY-Z

Componente	Significado	Valores Posibles
CP	Prefijo estándar de Caso de Prueba	CP
XX	Número del módulo (01-10)	01, 02, 03... 10
YY	Sub-caso dentro del módulo (01-99)	01, 02, 03...
Z	Capa de validación	F = Frontend, B = Backend, BD = Base de Datos

b. Ejemplo de nomenclatura

ID	Significado
CP-01-01-F	Módulo 01 (Incidencias), Sub-caso 01, validación en Frontend
CP-01-01-B	Módulo 01 (Incidencias), Sub-caso 01, validación en Backend
CP-03-02-BD	Módulo 03 (Responsables), Sub-caso 02, validación en Base de Datos
CP-09-01-F	Módulo 09 (Autenticación), Sub-caso 01, validación en Frontend

c. Principio de validación Dual

Cada funcionalidad debe ser probada en al menos dos capas para garantizar cobertura completa:

Capa	Propósito	Validaciones Típicas	Herramientas
Frontend (F)	Experiencia de usuario, feedback inmediato	Bloqueo de campos, mensajes de error visuales, formato de entrada, responsividad	Inspección manual, DevTools
Backend (B)	Seguridad, integridad de datos	Tipos de datos, rangos válidos, claves foráneas, transacciones, códigos HTTP	Postman, cURL
Base de Datos (BD)	Integridad estructural, consultas optimizadas	FK, normalización, triggers, procedimientos almacenados, consultas agregadas	MySQL Workbench, CLI

7. Matriz de caso de prueba

a. Modulo 01: Gestión de incidencias

Descripción: validación del ciclo completo de vida de una incidencia.

ID	Descripción	Capa	Pasos de Prueba	Datos de Prueba	Resultado Esperado	Responsable
CP - 01-	Formulario completo con todos	Frontend	1. Llenar título "Fuga de agua" 2. Llenar descripción	Título: "Fuga de agua" Descripción:	Botón se deshabilita, loading aparece,	Integrante 1

01-F	los campos válidos		"Tubería rota en calle principal" 3. Seleccionar prioridad "Alta" 4. Seleccionar ubicación (País→Provincia→Ciudad) 5. Seleccionar tipo/subtipo 6. Click en "Guardar"	"Tubería rota en calle principal" Prioridad: Alta	luego mensaje de éxito y redirección a lista	
CP - 01-01-B	Endpoint recibe y guarda datos correctamente	Backend	Enviar POST /api/incidencias con payload JSON válido	{ "titulo": "Fuga de agua", "descripcion": "Tubería rota...", "prioridad_id": 1, "ubicacion_id": 5, "tipo_id": 1, "subtipo_id": 3 }	HTTP 201, respuesta incluye id de incidencia creada, registros en tablas relacionadas	Integrante 2
CP - 01-02-F	Campo título vacío muestra error en UI	Frontend	1. Dejar campo título completamente vacío 2. Llenar demás campos obligatorios 3. Intentar guardar	Título: (vacío)	Mensaje "El campo título es obligatorio" aparece debajo del campo en color rojo, botón deshabilitado	Integrante 1
CP - 01-02-B	Endpoint rechaza título vacío con HTTP 422	Backend	Enviar POST /api/incidencias sin campo título	Payload sin campo "titulo"	HTTP 422, "errors": {"titulo": ["El campo título es obligatorio"]}	Integrante 2
CP - 01-03-F	Input teléfono solo acepta números	Frontend	1. Ir a campo teléfono 2. Intentar escribir letras "abc" 3. Escribir números "1234567890"	Intento: escribir "abc123"	Solo aparecen "123" en el campo, letras son bloqueadas inmediatamente	Integrante 1
CP - 01-	Backend valida formato	Backend	Enviar teléfono con letras via Postman	"telefono_contacto":	HTTP 422, "errors": {"telefono_contacto":	Integrante 2

03-B	teléfono con regex			"abc1234567890"	to": ["El formato del teléfono es inválido"]}	
CP - 01-04-F	Editar incidencia existente carga datos en formulario	Frontend	1. Ir a lista de incidencias 2. Click en botón "Editar" de una incidencia 3. Modificar título 4. Click en "Guardar"	Título original: "Fuga" Nuevo título: "Fuga de agua reportada"	Campos se precargan con datos actuales, cambios se reflejan en lista, mensaje de éxito	Integrante 1
CP - 01-04-B	Endpoint PUT actualiza correctamente en BD	Backend	Enviar PUT /api/incidencias/{id} con datos modificados	{ "titulo": "Fuga de agua reportada", ... }	HTTP 200, registro actualizado en tabla incidencias, timestamps modificados	Integrante 2
CP - 01-05-F	Modal de confirmación antes de eliminar	Frontend	1. Ir a incidencia específica 2. Click en botón "Eliminar" 3. No confirmar el modal	Click en eliminar	Modal aparece con mensaje "¿Está seguro de eliminar esta incidencia?" y botones "Cancelar" y "Eliminar"	Integrante 1
CP - 01-06-F	Eliminación exitosa tras confirmar modal	Frontend	1. Click en "Eliminar" 2. Confirmar en modal	Confirmar eliminación	Incidencia desaparece de la lista, toast/mensaje de éxito aparece	Integrante 1
CP - 01-06-B	Endpoint DELETE elimina lógicamente (soft delete)	Backend	Enviar DELETE /api/incidencias/{id}	-	HTTP 200, campo deleted_at actualizado, incidencia no aparece en consultas normales	Integrante 2

b. Modulo 02: Estados e historial

Descripción: Validación del flujo de cambios de estado y registro histórico completo.

ID	Descripción	Capa	Pasos de Prueba	Datos de Prueba	Resultado Esperado	Responsable
CP - 02-01-F	Dropdown muestra todos los estados disponibles	Frontend	1. Abrir incidencia existente 2. Localizar selector de estado	-	Dropdown muestra: Pendiente, En Proceso, Resuelto, Cerrado (según flujo definido)	Integrante 1
CP - 02-01-B	Endpoint retorna estados válidos desde BD	Backend	GET /api/estados	-	JSON: [{ "id": 1, "nombre": "Pendiente" }, { "id": 2, "nombre": "En Proceso" }, ...]	Integrante 2
CP - 02-02-F	Cambiar estado de Pendiente → En Proceso	Frontend	1. Incidencia con estado "Pendiente" 2. Seleccionar "En Proceso" del dropdown 3. Click en "Guardar Estado"	Estado actual: Pendiente Nuevo estado: En Proceso	Badge/color del estado cambia, pestaña historial se actualiza automáticamente	Integrante 1
CP - 02-02-B	PUT estado crea registro en historial con timestamp	Backend	PUT /api/incidencias/{id}/estado	{ "estado_id": 2, "comentario": "Iniciando revisión técnica" }	HTTP 200, nuevo registro en tabla historial_estados con usuario_id, fecha, estado_anterior, estado_nuevo	Integrante 2
CP - 02-03-F	Visualización de historial cronológico completo	Frontend	1. Ir a pestaña "Historial" de una incidencia 2. Ver lista de cambios	-	Lista ordenada cronológicamente (más reciente primero), muestra: estado anterior → nuevo, usuario responsable, fecha y hora	Integrante 1

CP - 02- 03- B	GET historial retorna datos ordenados por fecha	Backend	GET /api/incidencias/{id}/historial	-	Array JSON ordenado por created_at DESC, incluye: estado_origen, estado_destino, usuario, timestamp, comentario	Integrante 2
CP - 02- 04- F	Validación de flujo de estados (no permite estados inválidos)	Frontend	1. Ver opciones disponibles según estado actual 2. Intentar seleccionar un estado no permitido por flujo	Ejemplo: Intentar cambiar "Pendiente" → "Cerrado" si no está permitido	Opción no aparece en dropdown o aparece deshabilitada con tooltip "No permitido"	Integrante 1
CP - 02- 05- F	Fecha de resolución visible al marcar como Resuelto	Frontend	1. Cambiar estado a "Resuelto" 2. Observar datos mostrados	Estado: Resuelto	Campo "Fecha resolución: [dd/mm/aaaa hh:mm]" visible, tooltip muestra fecha completa	Integrante 1
CP - 02- 05- B	Fecha resolución guardada correctamente en BD	Backend	Consultar incidencia con estado "Resuelto"	-	Campo fecha_resolucion en tabla incidencias tiene timestamp válido, coincide con último cambio de estado	Integrante 3
CP - 02- 06- B	Trigger автоматически genera registro de historial	BD	Cambiar estado de incidencia via SQL	UPDATE incidencias SET estado_id = 3 WHERE id = 1	Nuevo registro insertado en tabla historial_estado s con todos los campos requeridos	Integrante 3

c. Módulo 03: Asignación responsable

Descripción: Validación de asignación de uno o varios usuarios con roles diferenciados.

ID	Descripción	Capa	Pasos de Prueba	Datos de Prueba	Resultado Esperado	Responsable
C P-03 -01 -F	Selector de usuarios con búsqueda/filtrado	Front end	1. Ir a sección "Asignar responsable" 2. Escribir parte del nombre en campo de búsqueda	"Juan"	Lista muestra únicamente usuarios que contienen "Juan" en nombre o apellido	Integrante 1
C P-03 -01 -B	Endpoint retorna usuarios filtrados por búsqueda	Back end	GET /api/usuarios?buscar=Juan	-	JSON con usuarios que coinciden: [{ "id": 5, "nombre": "Juan Pérez", ... }]	Integrante 2
C P-03 -02 -F	Asignar un responsable con rol específico	Front end	1. Seleccionar usuario "Juan Pérez" 2. Definir rol como "Responsable" 3. Click en "Asignar"	Usuario: Juan Pérez Rol: Responsable	Badge/pill muestra "Juan Pérez - Responsable" con color/icono diferenciado	Integrante 1
C P-03 -02 -B	Relación guardada en tabla pivote	Back end	POST /api/incidencias/{id}/responsables	{ "usuario_id": 5, "rol": "responsable" }	HTTP 200, nuevo registro en tabla incidencia_usuario con incidencia_id, usuario_id, rol, timestamps	Integrante 2
C P-03 -03 -F	Asignar múltiples responsables con diferentes roles	Front end	1. Asignar Juan como "Responsable" 2. Agregar María como "Apoyo" 3. Ver lista de responsables	Juan (Responsable), María (Apoyo)	Lista muestra ambos con roles diferenciados : badge verde para Responsable , badge azul para Apoyo	Integrante 1

C P- 03 - 03 -B	Múltiples registros en tabla pivote con roles diferentes	Back end	Verificar BD después de múltiples asignaciones	-	Tabla incidencia_usuario tiene 2 registros: uno con rol="responsable", otro con rol="apoyo", ambos con FK correctas	Integrante 3
C P- 03 - 04 -F	Cambiar rol de responsable existente	Front end	1. Ir a responsable "Juan" 2. Cambiar selector de rol a "Apoyo" 3. Guardar	Rol actual: Responsable Nuevo rol: Apoyo	Badge actualizado: "Juan - Apoyo", historial de cambio registrado	Integrante 1
C P- 03 - 04 -B	PUT actualiza rol en tabla pivote	Back end	PUT /api/incidencias/{id}/responsables/{usuario_id}	{ "rol": "apoyo" }	HTTP 200, campo rol actualizado, timestamp modificado	Integrante 2
C P- 03 - 05 -F	Eliminar responsable de incidencia	Front end	1. Click en icono "X" junto a responsable 2. Confirmar eliminación	-	Responsable removido de la lista, badge desaparece	Integrante 1
C P- 03 - 05 -B	DELETE/remover relación de tabla pivote	Back end	DELETE /api/incidencias/{id}/responsables/{usuario_id}	-	HTTP 200, registro eliminado de tabla pivote	Integrante 2

d. Modulo 04: Sistemas de comentarios

Descripción: Validación del registro y visualización de comentarios por incidencia.

ID	Descripción	Capa	Pasos de Prueba	Datos de Prueba	Resultado Esperado	Responsable
CP - 04-01-F	Agregar comentario con texto válido	Frontend	1. Ir a sección "Comentarios" de una incidencia 2. Escribir "Problema reportado al municipio" 3. Click en botón "Comentar"	Texto: "Problema reportado al municipio"	Comentario aparece en lista con: texto, autor (nombre del usuario logueado), fecha y hora relativa ("Hace 2 minutos")	Integrante 1
CP - 04-01-B	POST crea comentario con usuario y timestamps	Backend	POST /api/incidencias/{id}/comentarios	{ "texto": "Problema reportado al municipio" }	HTTP 201, registro creado en tabla comentarios con: id, incidencia_id, usuario_id, texto, created_at, updated_at	Integrante 2
CP - 04-02-F	Comentario vacío rechazado en frontend	Frontend	1. Dejar campo de texto completamente vacío 2. Intentar hacer click en "Comentar"	Texto: (vacío)	Botón "Comentar" permanece deshabilitado, o mensaje de error aparece al intentar enviar	Integrante 1
CP - 04-02-B	Backend rechaza texto vacío con validación	Backend	POST /api/incidencias/{id}/comentarios con texto vacío	{ "texto": "" }	HTTP 422, "errors": { "texto": ["El campo texto es obligatorio"] }	Integrante 2
CP - 04-	Contador de caracteres visible y funcional	Frontend	1. Escribir texto en textarea de comentarios 2. Observar contador	-	Contador muestra formato "X/1000" y cambia en	Integrante 1

03-F					tiempo real, cambia a rojo cuando se acerca al límite	
CP - 04-04-F	Ver comentarios ordenados por fecha (más reciente primero)	Frontend	1. Agregar comentario A 2. Esperar 10 segundos 3. Agregar comentario B 4. Ver lista	Comentario A: "Primero" Comentario B: "Segundo"	Comentario B aparece primero (más reciente), seguido de A	Integrante 1
CP - 04-04-B	GET retorna comentarios ordenados por created_at DESC	Backend	GET /api/incidencias/{id}/comentarios	-	Array JSON ordenado por created_at DESC , incluye datos completos del comentario	Integrante 2
CP - 04-05-F	Eliminar propio comentario	Frontend	1. Ver comentario creado por el usuario actual 2. Click en icono "Eliminar" del comentario 3. Confirmar	-	Comentario desaparece de la lista, toast de confirmación	Integrante 1
CP - 04-05-B	DELETE soft delete del comentario	Backend	DELETE /api/comentarios/{id}	-	HTTP 200, campo deleted_at actualizado, comentario no visible en consultas normales	Integrante 2

e. Modulo 05: Ubicación Georreferenciada

Descripción: Validación de selección jerárquica de ubicación (País → Provincia → Ciudad)

ID	Descripción	Capa	Pasos de Prueba	Datos de Prueba	Resultado Esperado	Responsable
----	-------------	------	-----------------	-----------------	--------------------	-------------

CP - 05 - 01 - F	Selección en cascada: país carga provincias	Frontend	1. Seleccionar país "Argentina" en dropdown 2. Esperar carga 3. Ver opciones del segundo dropdown	País: Argentina	Segundo dropdown (Provincias) se habilita y muestra opciones: Buenos Aires, Córdoba, Santa Fe, Mendoza, etc.	Integrante 1
CP - 05 - 01 - B	Endpoint retorna provincias filtradas por país	Backend	GET /api/provincias?pais_id=1	-	JSON con provincias del país seleccionado: [{"id": 1, "nombre": "Buenos Aires"}, ...]	Integrante 2
CP - 05 - 02 - F	Selección completa en cascada: País → Provincia → Ciudad	Frontend	1. Seleccionar "Argentina" 2. Seleccionar "Buenos Aires" 3. Esperar carga 4. Ver opciones de ciudades	País: Argentina Provincia: Buenos Aires	Tercer dropdown (Ciudades) se habilita y muestra: La Plata, Mar del Plata, Bahía Blanca, etc.	Integrante 1
CP - 05 - 02 - B	Endpoint retorna ciudades filtradas por provincia	Backend	GET /api/ciudades?provincia_id=1	-	JSON con ciudades de la provincia seleccionada	Integrante 2
CP - 05 - 03 - F	Cambio de provincia limpia ciudad seleccionada	Frontend	1. Seleccionar Argentina → Buenos Aires → La Plata 2. Cambiar provincia a "Córdoba" 3. Observar dropdown de ciudad	Ciudad previamente seleccionada: La Plata	Dropdown de ciudad se limpia automáticamente, muestra mensaje "Seleccione una ciudad"	Integrante 1
CP - 05 -	Backend valida relación provincia-	Backend	Intentar asignar ciudad_id de provincia diferente	ciudad_id: 100 (pertenece	HTTP 422, error de validación de	Integrante 2

03-B	ciudad en FK			a otra provincia)	integridad referencial	
CP - 05 - 04 - F	Campo ubicación muestra valor seleccionado completo	Frontend	1. Completar selección: Argentina / Buenos Aires / La Plata 2. Guardar incidencia 3. Ver en detalle	-	Campo ubicación muestra texto legible: "Argentina > Buenos Aires > La Plata"	Integrante 1
CP - 05 - 04 - B D	Tablas normalizadas sin redundancia de datos	BD	Verificar estructura de tablas de ubicación	-	Tablas separadas: países (id, nombre), provincias (id, nombre, pais_id), ciudades (id, nombre, provincia_id). FK correctas en cada nivel. Sin datos duplicados.	Integrante 3

f. Modulo 06: Clasificación Jerárquica.

Descripción: Validación de selección jerárquica de tipo y subtipo de incidencia

ID	Descripción	Capa	Pasos de Prueba	Datos de Prueba	Resultado Esperado	Responsable
CP - 06-01-F	Dropdown muestra tipos de incidencia activos	Frontend	1. Ir a campo "Tipo de Incidencia" 2. Abrir dropdown	-	Dropdown muestra opciones: Infraestructura, Seguridad, Servicios Públicos, Medio Ambiente, Otro	Integrante 1
CP - 06-	Endpoint retorna tipos ordenados	Backend	GET /api/tipos	-	JSON con tipos ordenados: [{"id": 1, "nombre":	Integrante 2

01-B	alfabéticamente				"Infraestructura", ...]	
CP - 06-02-F	Subtipo depende del tipo seleccionado (cascada)	Frontend	1. Seleccionar "Infraestructura" en campo Tipo 2. Esperar carga 3. Abrir dropdown de Subtipo	Tipo: Infraestructura	Dropdown de Subtipo se habilita y muestra: Alumbrado Público, Baches, Semáforos, Vallas, Drenaje	Integrante 1
CP - 06-02-B	Endpoint retorna subtipos filtrados por tipo	Backend	GET /api/subtipos?tipo_id=1	-	JSON con subtipos del tipo seleccionado: [{"id": 1, "nombre": "Alumbrado Público"}], ...]	Integrante 2
CP - 06-03-F	Cambio de tipo limpia subtipo seleccionado	Frontend	1. Seleccionar Infraestructura → Alumbrado Público 2. Cambiar Tipo a "Seguridad" 3. Observar Subtipo	Subtipo previamente seleccionado: Alumbrado Público	Campo Subtipo se limpia automáticamente, muestra "Seleccione un subtipo"	Integrante 1
CP - 06-03-B	Backend valida que subtipo pertenezca al tipo	Backend	Intentar asignar subtipo_id de tipo diferente	subtipo_id: 10 (pertenece a otro tipo)	HTTP 422, error de validación de relación jerárquica	Integrante 2
CP - 06-04-BD	Integridad referencial entre tipos y subtipos	BD	Verificar constraints de FK	-	Tabla subtipos tiene FK a tipos , no permite insertar subtipo con tipo_id inexistente	Integrante 3

g. Modulo 07: Sistemas de notificaciones

Descripción: Validación de creación y gestión de notificaciones del sistema

ID	Descripción	Capa	Pasos de Prueba	Datos de Prueba	Resultado Esperado	Responsable
----	-------------	------	-----------------	-----------------	--------------------	-------------

CP-07-01-F	Badge muestra contador de notificaciones no leídas	Frontend	1. Usuario tiene 3 notificaciones sin leer 2. Observar icono de campana en navbar	-	Badge/número rojo muestra "3" junto al icono de campana	Integrante 1
CP-07-02-F	Click en notificación la marca como leída	Frontend	1. Click en notificación no leída 2. Verificar badge	-	Badge decrementa (ejemplo: 3 → 2), notificación cambia estilo (fondo gris a blanco)	Integrante 1
CP-07-02-B	PATCH actualiza campo leído a true	Backend	PATCH /api/notificaciones/{id}	{ "leído": true }	HTTP 200, campo leído actualizado a true, timestamp leído_en registrado	Integrante 2
CP-07-03-F	Panel desplegable muestra lista de notificaciones	Frontend	1. Click en icono de campana 2. Ver panel	-	Panel se despliega con lista de notificaciones, cada una muestra: icono de tipo, mensaje resumido, tiempo relativo	Integrante 1
CP-07-04-B	Trigger/Evento crea notificación al cambiar estado	BD	1. Asignar incidencia a usuario 2. Cambiar estado de la incidencia	-	Tabla notificaciones tiene nuevo registro: usuario_id (destinatario), tipo (cambio estado), incidencia_id, leído=false, timestamps	Integrante 3
CP-07-05-F	Botón "Marcar todas como leídas" funciona	Frontend	1. Tener múltiples notificaciones sin leer 2. Click en "Marcar todas como leídas"	-	Todas las notificaciones cambian a estado leído, badge	Integrante 1

					desaparece o muestra 0	
CP-07-05-B	PATCH masivo actualiza todas las notificaciones	Backend	PATCH /api/notificaciones/marcar-leidas	-	HTTP 200, todas las notificaciones del usuario actual actualizan leido=true	Integrante 2

h. Módulo 08: Dashboard y Métricas

Descripción: Validación de visualización de métricas, gráficos y filtros del dashboard

ID	Descripción	Capa	Pasos de Prueba	Datos de Prueba	Resultado Esperado	Responsable
CP-08-01-F	Tarjeta principal muestra total de incidencias	Front end	1. Ir a dashboard 2. Observar tarjeta superior	-	Tarjeta muestra: "Total de Incidencias: 150" (número actualizado)	Integrante 1
CP-08-01-B	Endpoint retorna métricas generales agregadas	Back end	GET /api/metricas/generales	-	JSON: { "total": 150, "pendientes": 45, "en_proceso": 30, "resueltas": 75 }	Integrante 2
CP-08-02-F	Gráfico de barras muestra incidencias por estado	Front end	1. Ver sección de gráficos en dashboard	-	Gráfico de barras/torta visible con colores diferenciados por	Integrante 1

					estado, leyenda explicativa	
C P- 08 - 02 - B D	Query SQL agrupa correctamente por estado	BD	Ejecutar query de métricas	SELECT estado_id, COUNT(*) FROM incidencias GROUP BY estado_id	Resultado coincide con datos del dashboard, sin errores	Integrante 3
C P- 08 - 03 -F	Filtro por rango de fechas funciona	Front end	1. En dashboard, seleccionar fecha inicio: 01/06/2026 2. Seleccionar fecha fin: 08/06/2026 3. Click en "Aplicar"	Fecha inicio: 01/06/2026 Fecha fin: 08/06/2026	Tarjetas y gráficos muestran únicamente datos del rango seleccionado	Integrante 1
C P- 08 - 03 -B	Endpoint filtra por rango de fechas correctamente	Back end	GET /api/metricas/generales ?inicio=2026-06-01&fin=2026-06-08	-	JSON con métricas únicamente del rango especificado	Integrante 2
C P- 08 - 04 -F	Filtro por tipo muestra datos correctos	Front end	1. Seleccionar filtro "Tipo: Infraestructura" 2. Aplicar	Tipo: Infraestructura	Dashboard muestra únicamente incidencias de tipo Infraestructura	Integrante 1
C P- 08 - 04 -B	Query filtra por tipo_id correctamente	Back end	GET /api/metricas/generales ?tipo_id=1	-	Métricas filtradas por tipo_id = 1	Integrante 2
C P- 08	Filtro por ubicación	Front end	1. Seleccionar país/provincia/ciudad	Ubicación: Argentina > Buenos Aires	Dashboard muestra únicamente	Integrante 1

- 05 - F	muestra datos correctos		en filtros 2. Aplicar		te incidencia s de la ubicación seleccion ada	
C P- 08 - 05 - B	Query filtra por ubicación correcta mente	Back end	GET /api/metricas/generales ?ciudad_id=5	-	Métricas filtradas por ciudad_id = 5	Integran te 2
C P- 08 - 06 - B D	Tiempo promedio de resolución calculad o correcta mente	BD	Query tiempo promedio de resolución	SELECT AVG(DATEDIFF(fecha _resolucion, created_at)) FROM incidencias WHERE estado_id = 3 AND fecha_resolucion IS NOT NULL	Valor numérico en días (ejemplo: 3.5 días promedio)	Integran te 3

i. Módulo 09: Autenticación y Control de acceso

Descripción: Validación de login, logout y protección de rutas

ID	Descripción	Capa	Pasos de Prueba	Datos de Prueba	Resultado Esperado	Responsable
CP - 09 - 01 - F	Login con credenciales válidas redirige a dashboard	Fronte nd	1. Ingresar email: admin@ejemplo.com 2. Ingresar password: Admin123 3. Click en "Ingresar"	user: admin@ejemplo.com pass: Admin123	Spinner de carga, luego redirección a /dashboard, sin mensajes de error	Integrante 1
CP - 09 -	POST /login retorna token JWT o sesión	Backen d	POST /api/login	{ "email": "admin@ejemplo.com", "password": "Admin123" }	HTTP 200, respuesta incluye: { "token": "...", "user":	Integrante 2

01 -B					{ "id": 1, "nombre": "...", "rol": "admin" } }	
CP - 09 - 02 -F	Login con password incorrecto muestra error	Frontend	1. Credenciales correctas 2. Password: wrongpassword 3. Click "Ingresar"	pass: wrongpassword	Mensaje en pantalla: "Credenciales incorrectas. Verifique su email y contraseña." en color rojo	Integrante 1
CP - 09 - 02 -B	Login falla con credenciales inválidas retorna 401	Backend	POST /api/login con password incorrecto	-	HTTP 401, { "error": "Credenciales inválidas", "message": "Email o contraseña incorrectos" }	Integrante 2
CP - 09 - 03 -F	Login con email vacío muestra validación	Frontend	1. Dejar campo email vacío 2. Ingresar password 3. Click "Ingresar"	email: (vacío)	Mensaje debajo del campo: "El campo email es obligatorio"	Integrante 1
CP - 09 - 04 -F	Logout cierra sesión y redirige a login	Frontend	1. Click en "Cerrar sesión" del menú 2. Intentar acceder a /dashboard	-	Redirección a /login, sesión limpiada (token removido de localStorage)	Integrante 1
CP - 09 - 04 -B	POST /logout invalida token en servidor	Backend	POST /api/logout con header Authorization	Header: Bearer {token}	HTTP 200, token agregado a blacklist o sesión invalidada	Integrante 2

CP - 09 - 05 -F	Acceso sin autenticación redirige automáticamente a login	Frontend	1. Abrir nueva pestaña 2. Ir directamente a /dashboard sin login	-	Redirección automática a /login, URL cambia	Integrante 1
CP - 09 - 06 -F	Sesión expira y redirige a login	Frontend	1. Esperar expiración de token (o manipular localStorage) 2. Intentar hacer una request	Token expirado	Mensaje "Sesión expirada. Por favor, inicie sesión nuevamente.", redirección a /login	Integrante 1

j. Modulo 10: Validaciones de formato y tipo de datos

Descripción: Validación de login, logout y protección de rutas

ID	Descripción	Capa	Pasos de Prueba	Datos de Prueba	Resultado Esperado	Responsable
CP-10-01-F	Email con formato inválido muestra error en tiempo real	Frontend	1. Escribir "correo@" en campo email 2. Salir del campo (blur)	email: "correo@"	Mensaje de error aparece: "Ingrese un email válido (ejemplo: usuario@dominio.com)"	Integrante 1
CP-10-01-B	Backend valida formato email con regex	Backend	POST /api/login con email sin dominio	"email": "correo@"	HTTP 422, "errors": {"email": ["El formato del email es inválido"]}	Integrante 2
CP-10-	Campo descripción muestra contador de	Frontend	1. Ir a campo descripción de incidencia 2. Escribir texto	-	Contador visible: "0/500" que se actualiza al escribir, cambia a rojo al	Integrante 1

02-F	caracteres		3. Observar contador		superar 450 caracteres	
C P-10-03-F	Caracteres especiales HTML/XSS se sanitizan	Front end	1. Escribir en cualquier campo: <script>alert('hack')</script> 2. Guardar 3. Ver valor almacenado	Input: <script>alert('hack')</script>	Texto se muestra como texto plano, no como HTML interpretado. Verificar en DevTools que no haya código inyectado	Integrante 1
C P-10-03-B	Sanitización en backend previene XSS	Backend	Enviar payload con scripts en JSON	"descripcion": "<script>alert('xss')</script>"	HTTP 422 o texto almacenado sanitizado (caracteres HTML convertidos a entities)	Integrante 2
C P-10-04-F	Campo numérico no acepta letras	Front end	1. Ir a campo prioridad (dropdown o número) 2. Intentar escribir letras	-	Solo valores numéricos o opciones válidas del dropdown son aceptadas	Integrante 1
C P-10-05-F	Fecha inválida muestra mensaje de error	Front end	1. Ingresar fecha manualmente incorrecta 2. O seleccionar fecha imposible	fecha: "32/13/2026"	Mensaje de error: "Fecha inválida"	Integrante 1
C P-10-06-B	Backend valida rango de fechas permitidas	Backend	Enviar fecha fuera de rango válido	"fecha_creacion": "2030-01-01" (si no está permitido)	HTTP 422 según reglas de negocio, o accepted si la lógica lo permite	Integrante 2

8. Resumen de distribución de pruebas

a. Conteo por integrante y capa

Integrante	CP's Frontend (F)	CP's Backend (B)	CP's BD (BD)	Total CP's
Integrante 1 (Frontend)	46	0	0	46
Integrante 2 (Backend)	0	40	0	40
Integrante 3 (Base de Datos)	0	0	8	8
Total general	46	40	8	94

b. CP's por módulo

Módulo	Descripción	Total CP's
01	Gestión de Incidencias (CRUD)	12
02	Estados e Historial	11
03	Asignación de Responsables	10
04	Sistema de Comentarios	10
05	Ubicación Georreferenciada	9
06	Clasificación Jerárquica (Tipo/Subtipo)	8
07	Sistema de Notificaciones	10
08	Dashboard y Métricas	12
09	Autenticación	9
10	Validaciones de Formato	8

9. Plan de ejecución Pruebas

a. Cronograma Semanal

Semana	Días	Actividades	Entregables
Semana 1	1-2	Configurar entorno de testing (Postman, herramientas), definir matriz de CP's completa	Entorno listo, matriz CP v1.0
Semana 1	3-5	Ejecución de pruebas del Módulo 01 (CRUD) y Módulo 02 (Estados)	Capturas CP-01 y CP-02
Semana 2	1-3	Ejecución de pruebas del Módulo 03 (Responsables) y Módulo 04 (Comentarios)	Capturas CP-03 y CP-04
Semana 2	4-5	Ejecución de pruebas del Módulo 05 (Ubicación) y Módulo 06 (Tipo/Subtipo)	Capturas CP-05 y CP-06
Semana 3	1-3	Ejecución de pruebas del Módulo 07 (Notificaciones) y Módulo 08 (Dashboard)	Capturas CP-07 y CP-08
Semana 3	4-5	Ejecución de pruebas del Módulo 09 (Auth) y Módulo 10 (Validaciones)	Capturas CP-09 y CP-10
Semana 4	1-2	Pruebas de carga y estrés (JMeter/k6)	Reporte de JMeter
Semana 4	3-4	Verificación de integridad de BD, Laravel Pint	Reporte Pint, integridad BD
Semana 4	5	Consolidación de evidencias, tabla de métricas finales	Carpeta evidencias completa
Semana 5	1-2	Revisión final, ajustes finales, checklist pre-sustentación	Checklist completado
Semana 5	3-5	Preparación de documento técnico con evidencias	Documento técnico final

b. Criterios de progreso

- **Mínimo semanal:** 15 casos de prueba ejecutados con evidencia
- **Checkpoint:** Reunión de equipo cada viernes para revisar progreso y resolver bloqueos

- **Dependencia:** Los módulos más complejos (08, 09) requieren haber completado módulos base (01, 02)

10. Plantilla de Registro De ejecución de casos de prueba

a. Formato de registro

ID CP	Fecha Ejecución	Ejecutado Por	Módulo	Resultado	Severidad	Observaciones	Evidencia (Link/Captura)
CP-01-01-F	/06/2026		01	☐ Aprobado ☐ Fallido			
CP-01-01-B	/06/2026		01	☐ Aprobado ☐ Fallido			
CP-01-02-F	/06/2026		01	☐ Aprobado ☐ Fallido			
CP-01-02-B	/06/2026		01	☐ Aprobado ☐ Fallido			

b. Leyenda de severidad

Nivel	Descripción	Acción Requerida
Crítica	Bloquea funcionalidad principal	Corrección inmediata, re-test obligatorio
Alta	Afecta flujo importante	Corrección en máximo 24 horas
Media	Afecta funcionalidad secundaria	Corrección antes de entrega
Baja	Mejora cosmética o menor	Corrección si hay tiempo

11. Criterios De Cierre de plan de calidad

El plan de calidad se considera completo cuando se cumplen todos los siguientes criterios:

#	Criterio	Indicador	Estado
1	100% de casos de prueba ejecutados	Total ejecutados = Total planificados (94)	☐
2	100% de casos aprobados	Casos aprobados / Total ejecutados = 100%	☐
3	Validaciones frontend implementadas	Todos los CP-XX-F aprobados	☐
4	Validaciones backend implementadas	Todos los CP-XX-B aprobados	☐
5	Integridad de BD verificada	Todos los CP-XX-BD aprobados	☐
6	Pruebas de carga ejecutadas	Reporte JMeter/k6 generado	☐
7	Al menos 3 herramientas de calidad usadas	Postman + Pint + JMeter + DevTools ≥ 3	☐
8	Carpeta de evidencias completa	Todas las capturas guardadas y organizadas	☐
9	Tabla de métricas finales consolidada	Documento con valores obtenidos	☐
10	Ningún error JavaScript no controlado	Consola del navegador limpia	☐
11	Sin vulnerabilidades críticas OWASP identificadas	0 hallazgos críticos en reporte de auditoría OWASP	☐

12. Checklist Pre-Sustentación

#	Ítem	Descripción	Responsable	Estado
---	------	-------------	-------------	--------

1	Documento técnico	8-12 páginas, evidencias incluidas	Todos	☐
2	Carpeta evidencias	Estructura completa con todas las capturas	Todos	☐
3	Matriz CP completada	100% de casos ejecutados y documentados	Todos	☐
4	Capturas de validaciones	Frontend (UI) + Backend (Postman)	Integrante 1+2	☐
5	Reporte Laravel Pint	Verificación PSR-12 aplicada	Integrante 2	☐
6	Reporte JMeter/k6	Pruebas de carga documentadas	Integrante 3	☐
7	Credenciales de acceso	Usuario admin + usuario normal	Todos	☐
8	URL del sistema	Desplegado y accesible	Integrante 3	☐
9	Archivo SQL	Base de datos exportada	Integrante 3	☐
10	Código fuente	Repositorio con últimos cambios	Todos	☐

13. Métricas e indicadores del sistema.

Las siguientes métricas se medirán y documentarán como entregable final del plan (cubre O6):

Métrica	Descripción	Valor Objetivo	Herramienta de Medición	Estado
Tasa de validación backend	% de endpoints con validaciones activas	100%	Revisión de código / Postman	☐
Tasa de éxito de casos de prueba	% de casos funcionales aprobados	100%	Matriz de casos de prueba	☐
Tiempo promedio de respuesta	Latencia promedio bajo carga concurrente	< 2.0 seg	JMeter / k6	☐
Errores JS no controlados	Uncaught Errors en consola del navegador	0	DevTools del navegador	☐

Cobertura de evidencias	% de pruebas con captura o reporte adjunto	100%	Revisión del equipo	☐
Tiempo promedio de resolución	Días entre creación y estado "Resuelto"	Documentar valor real	Consulta SQL / Dashboard	☐
Cobertura funcional	Funcionalidades Validadas / Funcionalidades Totales	≥ 90%	Matriz de casos de prueba	☐
Densidad de defectos	Defectos Encontrados / Casos Ejecutados	< 0.1	Registro de ejecución de CP	☐
Vulnerabilidades OWASP críticas	Total hallazgos críticos en auditoría OWASP	0 críticas	Auditoría OWASP manual / ZAP	☐

14. Riesgos de calidad y acciones mitigadoras

Riesgo Técnico	Impacto	Probabilidad	Acción Preventiva (QA)	Acción Correctiva (QC)
Pérdida de datos en contenedores	Crítico	Baja	Configurar volúmenes locales (volumes:) en docker-compose.yml	Backup manual antes de reiniciar contenedores
Bloqueo por CORS	Alto	Media	Instalar middleware CORS de Laravel, whitelist del frontend	Revisar configuración en config/cors.php
Inyección de datos corruptos	Crítico	Baja	Reglas estrictas en Laravel Form Requests: required, integer, exists:ciudades,id	Limpiar BD, re-validar endpoints con Postman
Redundancia en ubicación	Alto	Media	Aplicar normalización 3FN con tablas separadas: paises, provincias, ciudades	Reestructurar BD si se detecta redundancia
Evidencias incompletas	Alto	Media	Política de equipo: documentar cada prueba al ejecutarla, no al final	Sesión dedicada de capturas antes de entrega

Sesión expira durante pruebas	Medio	Media	Configurar tiempo de expiración adecuado, token refresh	Regenerar token, re-ejecutar pruebas
Errores funcionales en flujo de estados	Alta	Alta	Diseñar casos de prueba específicos para cada transición de estado; validar flujo en CP-02-*	Rollback manual de estado; re-test completo del módulo de estados
Consultas lentas en mapa dinámico	Media	Baja	Limitar registros en consultas geográficas; usar paginación y caché en endpoints de mapa	Optimizar queries SQL; agregar índices a campos de coordenadas

15. Herramientas

Categoría	Herramienta	Propósito	Objetivo Cubierto	Acción Correctiva (QC)
Calidad de Código	Laravel Pint	Verificación automática del estándar PSR-12 en código PHP	O5	Backup manual antes de reiniciar contenedores
Pruebas de API	Postman	Validación de endpoints REST, códigos HTTP, payloads JSON	O1, O3, O5	Revisar configuración en config/cors.php
Pruebas de Carga	Apache JMeter / k6	Simulación de usuarios concurrentes y medición de latencia	O4, O5	Limpiar BD, re-validar endpoints con Postman
Inspección de UI	DevTools del navegador	Auditoría de consola JavaScript, red, responsividad	O1, O3	Reestructurar BD si se detecta redundancia
Orquestación	Docker Compose	Despliegue, aislamiento y verificación de persistencia del entorno	O4	Sesión dedicada de capturas antes de entrega
Control de Versiones	Git / GitHub	Gestión de ramas, code reviews, trazabilidad de cambios	O5	Regenerar token, re-ejecutar pruebas

